

EPPS 6354 Information Management
Cyber Operations Incident Database (COID):
A Normalized Relational Database for Cross-National Analysis of
State-Sponsored Cyber Activity

<https://github.com/JingUSA/COID>

<https://lite.datasette.io/?url=https://raw.githubusercontent.com/JingUSA/COID/main/data/coid.db#/coid>

Jing Tao
May 2026

1 Introduction

State-sponsored cyber operations have emerged as a defining feature of contemporary international conflict. Despite growing scholarly attention, systematic empirical analysis remains constrained by the absence of purpose-built research infrastructure. This project addresses that gap by designing and implementing the Cyber Operations Incident Database (COID), a normalized relational database built from the Council on Foreign Relations (CFR) Cyber Operations Tracker. The raw source data contains 2,142 records representing 792 unique incidents spanning August 2005 through March 2023, involving 58 sponsoring states and 119 victim countries.

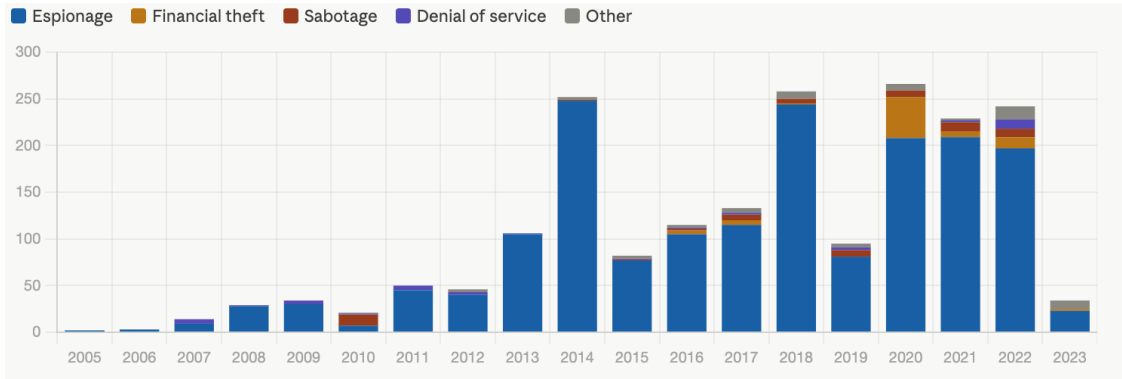


Figure 1: Annual disclosed cyber incidents by operation type, 2005–2023. Source: CFR Cyber Operations Tracker.

Espionage dominates across all years (88.3 percent of typed records). Figure 2 disaggregates volume by the five most active sponsors. China leads overall; Russia spikes in 2014 (Crimea) and 2022 (Ukraine); Iran peaks in 2018; North Korea escalates sharply from 2020 onward.

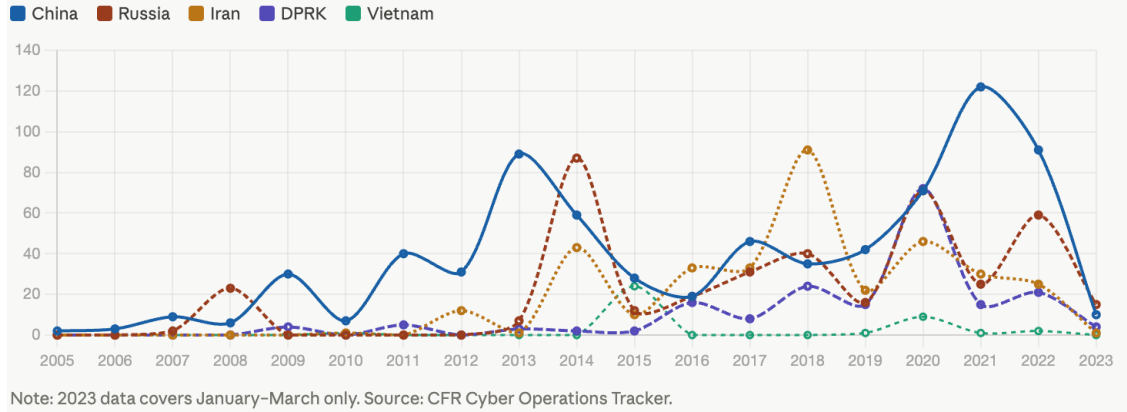


Figure 2: Annual disclosed cyber incidents by top five sponsor states, 2005–2023. Source: CFR Cyber Operations Tracker.

The raw CFR data is a denormalized flat file in which one incident appears in multiple rows when it targets more than one country or sector. COID resolves this by decomposing the flat file into five normalized tables, enforcing referential integrity through primary key and foreign key constraints, and exposing a SQL query layer that supports reproducible analytical reporting. The database is designed to answer four research questions: (1) which countries are most frequently targeted; (2) which sponsors are most active and what are the dominant bilateral pairings; (3) how operational types are distributed across victim sectors; and (4) what is the cumulative growth trajectory of disclosed incidents over time. The entity-relationship model, relational schema, DDL, ETL pipeline, and query library are all original contributions of this project.

2 Entity-Relationship Model

The entity-relationship (ER) model is a conceptual representation of the data structures required to support the research questions defined in Section 1. It identifies the real-world entities to be stored in the database, the attributes that describe each entity, and the relationships between them. The ER model serves as the blueprint from which the relational schema in Section 3 is derived. No SQL is written at this stage; the purpose is to reason about structure before implementation.

The CFR Cyber Operations Tracker raw data consists of 2,142 rows in a single denormalized flat file. Careful inspection of the 15 source columns reveals five distinct real-world entities: *country*, *incident*, *type*, *category*, and *incident_target*. These five entities and the relationships among them constitute the ER model for the Cyber Operations Incident Database (COID).

2.1 Entities and Attributes

country. The *country* entity represents a sovereign state. Countries appear in the raw data in two distinct roles: as the sponsoring state of a cyber operation (the **Sponsor** column) and as the victim state of a cyber operation (the **COUNTRY_ALPHA** column). Because a country is the same real-world entity regardless of its role in any particular operation, a single *country* table serves both purposes. The primary key is **country_**

alpha3, the ISO 3166-1 alpha-3 three-character country code, which is unique and already present in the raw data for victim countries. The only additional attribute is **country_name**, the full country name.

incident. The *incident* entity represents one discrete state-sponsored cyber operation as recorded by the CFR Tracker. The raw data contains 2,142 rows but only 792 unique operation titles; the same incident appears in multiple rows when it targeted more than one country or sector. The *incident* table stores one row per unique operation. Its primary key is **incident_id**, a system-generated serial integer. Descriptive attributes include **title**, **incident_date**, **description**, **affiliations**, and **response**. The three source URL columns from the raw data (**Sources_1**, **Sources_2**, **Sources_3**) are concatenated into a single **sources** text field during ETL. The foreign key **sponsor_country** references **country.country_alpha3** and identifies the state responsible for the operation.

type. The *type* entity is a dimension table representing the seven operational categories recorded in the **Type** column of the raw data: Espionage, Financial Theft, Sabotage, Denial of Service, Data Destruction, Defacement, and Doxing. Storing these values in a dedicated table with a foreign key constraint, rather than as free text in a fact table, prevents data entry errors and enforces domain integrity. The primary key is **type_id** (serial integer) and the only descriptive attribute is **type_name**.

category. The *category* entity is a dimension table representing the four target sector categories recorded in the **Category** column: Government, Private sector, Military, and Civil society. The design rationale is identical to that of the *type* table. The primary key is **category_id** and the descriptive attribute is **category_name**.

incident_target. The *incident_target* entity represents a single targeting record: for a given operation, it captures which country was targeted, by what method, in which sector, and which specific organizations were affected. Each row in the original 2,142-row flat file corresponds to exactly one row in *incident_target*. In one sentence: each row answers the question of which country a given operation struck, in which sector, by which method, and against which specific victims. Because one operation can target multiple countries and one country can be targeted by multiple operations, the relationship between *incident* and *country* (in the victim role) is many-to-many; *incident_target* is the bridge table that resolves this relationship. Its primary key is **target_id**, a surrogate serial integer. Foreign keys reference *incident*, *country*, *type*, and *category*. The **victims** text field preserves the names of the specific organizations affected, as recorded in the raw data.

Table 1 summarizes all five entities and their attributes.

Table 1: COID entities and attributes.

Entity	Attribute	Data Type	Role	Description
country	country_alpha3	CHAR(3)	PK	ISO 3166-1 alpha-3 code
country	country_name	VARCHAR(100)		Full country name
incident	incident_id	SERIAL	PK	Auto-generated identifier
incident	title	VARCHAR(300)		Operation name
incident	incident_date	DATE		Date of disclosure
incident	description	TEXT		Narrative description
incident	affiliations	TEXT		Associated threat actor groups
incident	response	TEXT		Victim response
incident	sources	TEXT		Combined source URLs
incident	sponsor_country	CHAR(3)	FK → country	Sponsoring state (NULL if multi-sponsor)
type	type_id	SERIAL	PK	Auto-generated identifier
type	type_name	VARCHAR(50)		Operation type label (7 values)
category	category_id	SERIAL	PK	Auto-generated identifier
category	category_name	VARCHAR(50)		Target sector label (4 values)
incident_target	target_id	SERIAL	PK	Surrogate key
incident_target	incident_id	INTEGER	FK → incident	References parent operation
incident_target	country_alpha3	CHAR(3)	FK → country	Victim country
incident_target	type_id	INTEGER	FK → type	Operation type (NULL ok: 108 records)
incident_target	category_id	INTEGER	FK → category	Target sector (NULL ok: 27 records)
incident_target	victims	TEXT		Victim organization names

2.2 Relationships and Cardinality

Five relationships connect the five entities. Each relationship is defined by its parent entity (the “one” side), its child entity (the “many” side), and its cardinality, which is derived directly from the structure of the raw data.

The *sponsors* relationship connects *country* to *incident*: one country can sponsor many operations, but each operation has at most one sponsoring country recorded. This is a one-to-many (1:N) relationship, implemented as the foreign key `incident.sponsor_country`.

The *has* relationship connects *incident* to *incident_target*: one operation can produce many targeting records, one for each country-sector combination it affected. This is a 1:N relationship implemented as `incident_target.incident_id`.

The *targeted by* relationship connects *country* to *incident_target*: one country can appear as a victim in many targeting records across many operations. This is a 1:N relationship implemented as `incident_target.country_alpha3`. Together, the *has* and *targeted by* relationships decompose the many-to-many relationship between *incident* and *country* (victim role) through the *incident_target* bridge table.

The two *classifies* relationships connect *type* and *category* to *incident_target* respectively: one type label can classify many targeting records, and one category label can classify many targeting records. Both are 1:N relationships implemented as `incident_target.type_id` and `incident_target.category_id`.

Table 2 summarizes all five relationships.

Table 2: COID entity relationships and cardinality.

Relationship	Parent	Child	Cardinality	FK Column	Basis in Raw Data
sponsors	country	incident	1:N	incident.sponsor_country	One Sponsor value per row
has	incident	incident_target	1:N	incident_target.incident_id	One operation maps to multiple target rows
targeted by	country	incident_target	1:N	incident_target.country_alpha3	One country appears across multiple target rows
classifies	type	incident_target	1:N	incident_target.type_id	One Type value per row
classifies	category	incident_target	1:N	incident_target.category_id	One Category value per row

2.3 Design Decisions

Decision 1: Attacker and victim countries share one table. The *country* table serves both the sponsor role (via `incident.sponsor_country`) and the victim role (via `incident_target.country_alpha3`). An alternative design would maintain separate *sponsor_country* and *victim_country* tables. This alternative was rejected because a country is the same real-world entity regardless of its role in any particular operation: China appears as both a prolific sponsor and an occasional victim in the CFR data. Storing the same country twice in two separate tables would introduce redundancy and violate the first normal form principle that each fact should be stored exactly once. A single *country* table referenced by two foreign keys from different tables cleanly separates role from entity.

Decision 2: Type and category are stored as dimension tables, not as free text. An alternative design would store `type_name` and `category_name` as VARCHAR columns directly inside *incident_target*, avoiding the need for *type* and *category* tables and their associated foreign keys. This alternative was rejected for two reasons. First, free-text storage provides no mechanism to prevent inconsistent entries. Second, the fixed and known value sets for both fields (seven types, four categories) are precisely the conditions under which a dimension table with a foreign key constraint is the appropriate design. The constraint guarantees that only valid values can be inserted, which is a fundamental data quality requirement for a research database.

Decision 3: incident_target uses a surrogate primary key. The initial design used a composite primary key of (`incident_id`, `country_alpha3`). Inspection of the raw data revealed one case in which the same operation targeted the same country with two different operation types, producing two rows with identical `incident_id` and `country_alpha3` values but different `type_id` values. A composite key on those two columns would reject the second row as a duplicate. The surrogate key `target_id SERIAL` avoids this constraint violation while still allowing the data to be queried correctly.

Figure 3 presents the complete entity-relationship diagram for COID.

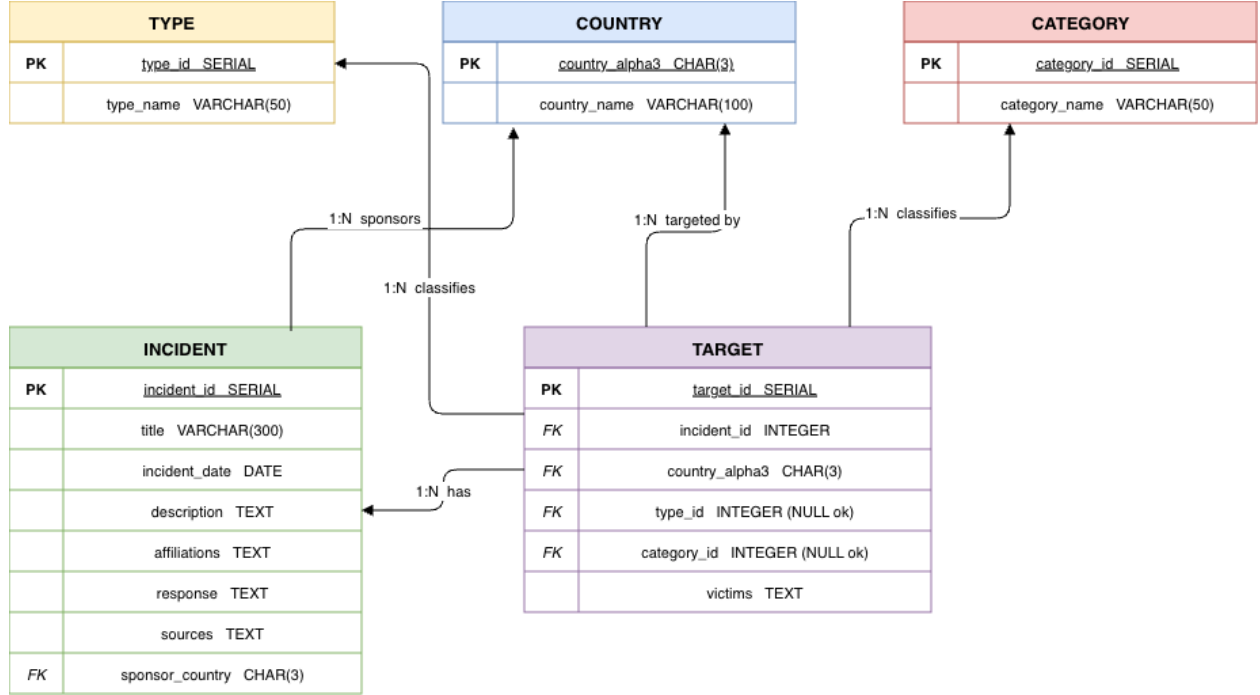


Figure 3: Entity-relationship diagram for the Cyber Operations Incident Database (COID). PK = primary key; FK = foreign key. Underlined attributes denote primary keys. Crow’s foot notation is used for cardinality.

3 Relational Schema

The relational schema translates the entity-relationship model into a concrete set of table definitions that can be implemented in a database management system. Each entity in the ER model becomes a table; each attribute becomes a column with a specified data type; and each relationship is implemented through primary key and foreign key constraints. The schema for COID consists of five tables: *country*, *incident*, *type*, *category*, and *incident_target*. All tables are implemented in PostgreSQL 18.

Three categories of constraints are applied throughout the schema. Primary key constraints enforce uniqueness and prohibit null values on identifier columns. Foreign key constraints enforce referential integrity, ensuring that every value in a child column references an existing value in the corresponding parent column. Check constraints enforce domain integrity by restricting column values to a defined set of valid entries. Together these constraints ensure that the database cannot enter an inconsistent state through ordinary insert or update operations.

3.1 Table Definitions

country. The *country* table is the shared dimension for all geographic references in COID. It is populated first during ETL because all other tables that reference a country depend on this table existing and being complete.

```
CREATE TABLE country (
    country_alpha3 CHAR(3)          PRIMARY KEY,
    country_name   VARCHAR(100) NOT NULL
);
```

The primary key `country_alpha3` is the ISO 3166-1 alpha-3 code. It is a natural key chosen over a surrogate integer because the three-letter code is already present and consistently formatted in the raw data, is human-readable in query results, and is a stable international standard that is unlikely to change. `country_name` is declared `NOT NULL` because every country record must have a recognizable name.

type. The *type* table stores the seven operation type labels as a controlled vocabulary.

```
CREATE TABLE type (
    type_id   SERIAL          PRIMARY KEY,
    type_name VARCHAR(50) NOT NULL UNIQUE
);
```

The `UNIQUE` constraint on `type_name` prevents duplicate labels from being inserted. The seven valid values inserted during ETL are: Espionage, Financial Theft, Sabotage, Denial of service, Data destruction, Defacement, and Doxing.

category. The *category* table stores the four target sector labels as a controlled vocabulary.

```
CREATE TABLE category (
    category_id   SERIAL          PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL UNIQUE
);
```

The four valid values inserted during ETL are: Government, Private sector, Military, and Civil society.

incident. The *incident* table stores one row per unique cyber operation. It is the central fact table of COID and the primary unit of analysis for research questions about operation-level attributes.

```
CREATE TABLE incident (
    incident_id   SERIAL          PRIMARY KEY,
    title         VARCHAR(300) NOT NULL,
    incident_date DATE,
    description   TEXT,
    affiliations  TEXT,
    response      TEXT,
    sources       TEXT,
    sponsor_country CHAR(3)
                    REFERENCES country(country_alpha3)
                    ON DELETE SET NULL
);
```

`incident_date` permits null values because a small number of CFR records carry only a year or an approximate date that cannot be reliably parsed to a full `DATE` value during ETL. `sponsor_country` is nullable because 28 records in the raw data carry a multi-state sponsor value (e.g., “United Kingdom, United States”) that cannot be mapped to a single `country_alpha3`; these records receive a null sponsor during ETL and are documented as a data limitation. The `ON DELETE SET NULL` clause ensures that if a country record were ever removed from the *country* table, the referencing incident row is not deleted but instead has its `sponsor_country` set to null.

incident_target. The *incident_target* table stores one row for each targeting record in the raw data. It is the bridge table that resolves the many-to-many relationship between *incident* and *country*, and it links each targeting event to its operation type and target sector.

```
CREATE TABLE incident_target (
    target_id      SERIAL      PRIMARY KEY,
    incident_id    INTEGER     NOT NULL
                        REFERENCES incident(incident_id)
                        ON DELETE CASCADE,
    country_alpha3 CHAR(3)     NOT NULL
                        REFERENCES country(country_alpha3),
    type_id        INTEGER
                        REFERENCES type(type_id),
    category_id    INTEGER
                        REFERENCES category(category_id),
    victims        TEXT
);
```

`incident_id` and `country_alpha3` are declared `NOT NULL` because every targeting record must be associated with a known operation and a known victim country. `type_id` and `category_id` are nullable because 108 records in the raw data carry no `Type` value and 27 carry no `Category` value respectively. The `ON DELETE CASCADE` clause on `incident_id` ensures that if a parent incident row is deleted, all associated targeting records are automatically removed, preserving referential integrity. A surrogate primary key `target_id` is used rather than a composite key on (`incident_id`, `country_alpha3`) because one incident in the raw data targets the same country under two different operation types, which would violate a composite uniqueness constraint on those two columns alone.

3.2 Indexes

Four additional indexes are defined on columns that appear frequently in `WHERE` clauses, `JOIN` conditions, and `GROUP BY` expressions in the representative queries presented in Section 6. Indexes accelerate query execution by allowing the query planner to locate rows without scanning the full table.

```
CREATE INDEX idx_incident_date
    ON incident (incident_date);

CREATE INDEX idx_incident_sponsor
```

```

ON incident (sponsor_country);

CREATE INDEX idx_target_incident
ON incident_target (incident_id);

CREATE INDEX idx_target_country
ON incident_target (country_alpha3);

```

`idx_incident_date` supports time-series queries that filter or group operations by year. `idx_incident_sponsor` supports queries that aggregate incident counts by sponsoring state. `idx_target_incident` and `idx_target_country` support the join between *incident_target* and its two most frequently referenced parent tables.

3.3 Schema Summary

Table 3 provides a consolidated reference for the complete COID relational schema, including all columns, data types, and constraint specifications.

Table 3: COID relational schema.

Table.	Data Type	Constraints	Notes
country.country_alpha3	CHAR(3)	PRIMARY KEY	ISO alpha-3 code
country.country_name	VARCHAR(100)	NOT NULL	Full country name
type.type_id	SERIAL	PRIMARY KEY	Auto-generated
type.type_name	VARCHAR(50)	NOT NULL, UNIQUE	7 controlled values
category.category_id	SERIAL	PRIMARY KEY	Auto-generated
category.category_name	VARCHAR(50)	NOT NULL, UNIQUE	4 controlled values
incident.incident_id	SERIAL	PRIMARY KEY	Auto-generated
incident.title	VARCHAR(300)	NOT NULL	Operation name
incident.incident_date	DATE		NULL if date unclear
incident.description	TEXT		
incident.affiliations	TEXT		Threat actor groups
incident.response	TEXT		Victim response
incident.sources	TEXT		Source URLs
incident.sponsor_country	CHAR(3)	FK → country, ON DELETE SET NULL	NULL if multi-sponsor
incident_target.target_id	SERIAL	PRIMARY KEY	Surrogate key
incident_target.incident_id	INTEGER	NOT NULL, FK → incident, ON DELETE CASCADE	
incident_target.country_alpha3	CHAR(3)	NOT NULL, FK → country	Victim country
incident_target.type_id	INTEGER	FK → type	NULL ok: 108 rows
incident_target.category_id	INTEGER	FK → category	NULL ok: 27 rows
incident_target.victims	TEXT		Victim org names

Figure 4 presents the complete schema structure for all five tables as reported by the PostgreSQL `\d` output for all five COID tables as executed in the macOS terminal, showing column definitions, data types, constraints, and indexes as implemented in PostgreSQL 18.3 via Postgres.app.

```
(base) jingtao@jings-air ~ % psql -h localhost -p 5433 -U jingtao -d coid -c "SELECT incident_id, title, incident_date, sponsor_country FROM incident LIMIT 5;"
incident_id | title | incident_date | sponsor_country
-----
1 | Titan Rain | 2005-08-25 | CHN
2 | Compromise at the State Department | 2006-07-11 | CHN
3 | Compromise of the Pentagon's NIPRNet | 2006-08-17 | CHN
4 | Compromise at U.S. Naval War College | 2006-12-04 | CHN
5 | Compromise of National Defense University | 2007-01-12 | CHN
(5 rows)

(base) jingtao@jings-air ~ % \d+
zsh: command not found: d+
(base) jingtao@jings-air ~ % psql -h localhost -p 5433 -U jingtao -d coid -c "\d+ country" -c "\d+ type" -c "\d+ category" -c "\d+ incident" -c "\d+ incident_target"

Table "public.country"
Column | Type | Collation | Nullable | Default | Storage | Compression | Stats target | Description
-----
country_alpha3 | character(3) | | not null | | extended | | | 
country_name | character varying(100) | | not null | | extended | | | 
Indexes:
    "country_pkey" PRIMARY KEY, btree (country_alpha3)
Referenced by:
    TABLE "incident" CONSTRAINT "incident_sponsor_country_fkey" FOREIGN KEY (sponsor_country) REFERENCES country(country_alpha3) ON DELETE SET NULL
    TABLE "incident_target" CONSTRAINT "incident_target_country_alpha3_fkey" FOREIGN KEY (country_alpha3) REFERENCES country(country_alpha3)
Not-null constraints:
    "country_country_alpha3_not_null" NOT NULL "country_alpha3"
    "country_country_name_not_null" NOT NULL "country_name"
Access method: heap

Table "public.type"
Column | Type | Collation | Nullable | Default | Storage | Compression | Stats target | Description
-----
type_id | integer | | not null | nextval('type_type_id_seq'::regclass) | plain | | | 
type_name | character varying(50) | | not null | | extended | | | 
Indexes:
    "type_pkey" PRIMARY KEY, btree (type_id)
    "type_type_name_key" UNIQUE CONSTRAINT, btree (type_name)
Referenced by:
    TABLE "incident_target" CONSTRAINT "incident_target_type_id_fkey" FOREIGN KEY (type_id) REFERENCES type(type_id)
Not-null constraints:
    "type_type_id_not_null" NOT NULL "type_id"
    "type_type_name_not_null" NOT NULL "type_name"
Access method: heap

Table "public.category"
Column | Type | Collation | Nullable | Default | Storage | Compression | Stats target | Description
-----
category_id | integer | | not null | nextval('category_category_id_seq'::regclass) | plain | | | 
category_name | character varying(50) | | not null | | extended | | | 
Indexes:
    "category_pkey" PRIMARY KEY, btree (category_id)
    "category_category_name_key" UNIQUE CONSTRAINT, btree (category_name)
Referenced by:
    TABLE "incident_target" CONSTRAINT "incident_target_category_id_fkey" FOREIGN KEY (category_id) REFERENCES category(category_id)
Not-null constraints:
    "category_category_id_not_null" NOT NULL "category_id"
    "category_category_name_not_null" NOT NULL "category_name"
Access method: heap

Table "public.incident"
Column | Type | Collation | Nullable | Default | Storage | Compression | Stats target | Description
-----
incident_id | integer | | not null | nextval('incident_incident_id_seq'::regclass) | plain | | | 
title | character varying(300) | | not null | | extended | | | 
incident_date | date | | not null | | plain | | | 
description | text | | | | extended | | | 
affiliations | text | | | | extended | | | 
response | text | | | | extended | | | 
sources | text | | | | extended | | | 
sponsor_country | character(3) | | | | extended | | | 
Indexes:
    "incident_pkey" PRIMARY KEY, btree (incident_id)
    "idx_incident_date" btree (incident_date)
    "idx_incident_sponsor" btree (sponsor_country)
Foreign-key constraints:
    "incident_sponsor_country_fkey" FOREIGN KEY (sponsor_country) REFERENCES country(country_alpha3) ON DELETE SET NULL
Referenced by:
    TABLE "incident_target" CONSTRAINT "incident_target_incident_id_fkey" FOREIGN KEY (incident_id) REFERENCES incident(incident_id) ON DELETE CASCADE
Not-null constraints:
    "incident_incident_id_not_null" NOT NULL "incident_id"
    "incident_title_not_null" NOT NULL "title"
Access method: heap

Table "public.incident_target"
Column | Type | Collation | Nullable | Default | Storage | Compression | Stats target | Description
-----
target_id | integer | | not null | nextval('incident_target_target_id_seq'::regclass) | plain | | | 
incident_id | integer | | not null | | plain | | | 
country_alpha3 | character(3) | | not null | | extended | | | 
type_id | integer | | | | plain | | | 
category_id | integer | | | | plain | | | 
victims | text | | | | extended | | | 
Indexes:
    "incident_target_pkey" PRIMARY KEY, btree (target_id)
    "idx_target_country" btree (country_alpha3)
    "idx_target_incident" btree (incident_id)
Foreign-key constraints:
    "incident_target_category_id_fkey" FOREIGN KEY (category_id) REFERENCES category(category_id)
    "incident_target_country_alpha3_fkey" FOREIGN KEY (country_alpha3) REFERENCES country(country_alpha3)
    "incident_target_incident_id_fkey" FOREIGN KEY (incident_id) REFERENCES incident(incident_id) ON DELETE CASCADE
    "incident_target_type_id_fkey" FOREIGN KEY (type_id) REFERENCES type(type_id)
Not-null constraints:
    "incident_target_target_id_not_null" NOT NULL "target_id"
```

Figure 4: PostgreSQL \d+ output for all five COID tables, showing column definitions, data types, constraints, and indexes as implemented in PostgreSQL 18.3.

4 ETL Pipeline

ETL stands for Extract, Transform, and Load. The ETL pipeline is the process by which the raw CFR Cyber Operations Tracker source file is extracted from its original format, transformed into a structure consistent with the normalized relational schema defined in Section 3, and loaded into the five tables of the COID database. All ETL operations are implemented in Python 3.12 using the `pandas` library for data manipulation and the `psycopg2` library for PostgreSQL connectivity. The pipeline connects to the COID database running on PostgreSQL 18.3 via Postgres.app on a local machine at port 5433.

The raw source file is an Excel workbook containing 2,142 rows and 15 columns. It is loaded into a `pandas` DataFrame using `read_excel` with the `openpyxl` engine. No pre-processing or merging is performed before loading; the pipeline reads the raw file exactly as distributed by the CFR and performs all transformations programmatically.

4.1 Stage 1: Dimension Table Population

The two dimension tables with fixed controlled vocabularies, *type* and *category*, are populated first using static `INSERT` statements executed directly in the PostgreSQL session. The seven type values (Espionage, Financial Theft, Sabotage, Denial of service, Data destruction, Defacement, Doxing) and four category values (Government, Military, Private sector, Civil society) are inserted before any fact data is loaded, as the foreign key constraints on *incident_target* require these parent rows to exist.

4.2 Stage 2: Country Population

The *country* table is populated by extracting all distinct `COUNTRY_ALPHA` and `victims_country` pairs from the raw data. The `COUNTRY_ALPHA` column provides the ISO alpha-3 primary key value; the `victims_country` column provides the associated country name. Sponsor countries that do not appear as victim countries are added separately using a manually constructed mapping dictionary that translates sponsor name strings (e.g., “China”, “Russian Federation”) to their corresponding ISO alpha-3 codes. All inserts use `ON CONFLICT DO NOTHING` to handle duplicate country codes gracefully. A total of 119 country records are inserted.

4.3 Stage 3: Incident Population

The *incident* table is populated from a deduplicated view of the raw data. Because the same incident appears in multiple rows when it targeted multiple countries or sectors, the raw DataFrame is first deduplicated on the `Title` column, retaining the first occurrence of each unique title. This produces 792 unique incident records.

For each incident, the following transformations are applied. The `Date` column is parsed to a Python `date` object using `pandas.to_datetime`; rows where the date cannot be parsed are assigned a null value. The three source URL columns (`Sources_1`, `Sources_2`, `Sources_3`) are concatenated into a single string separated by pipe characters and stored in the `sources` column. The `Sponsor` column is mapped to an ISO alpha-3 code using the country mapping dictionary; records where the sponsor string contains a comma

(indicating a multi-state attribution, 28 records) are assigned a null `sponsor_country` value. Each incident row is inserted individually using a parameterized `INSERT ... RETURNING incident_id` statement, and the returned identifier is stored in a Python dictionary mapping title strings to database-assigned incident IDs.

4.4 Stage 4: Incident_target Population

The *incident_target* table is populated by iterating over all 2,142 rows of the original raw DataFrame. For each row, the incident ID is retrieved from the title-to-ID dictionary constructed in Stage 3. The `COUNTRY_ALPHA` value provides the victim country foreign key. The `Type` and `Category` values are mapped to their corresponding integer IDs using lookup dictionaries constructed from the *type* and *category* tables. Rows where `COUNTRY_ALPHA` is null (24 records) are skipped. All valid target rows are inserted in a single batch using `execute_values` for efficiency.

4.5 Verification

Following completion of the ETL pipeline, row counts are verified against expected values. Table 4 presents the expected and actual row counts for all five tables.

Table 4: ETL verification: expected and actual row counts.

Table	Expected	Actual	Notes
country	119	119	All victim and sponsor countries
type	7	7	Fixed controlled vocabulary
category	4	4	Fixed controlled vocabulary
incident	792	792	Unique operations by title
incident_target	2118	2118	24 rows skipped due to null country

Figure 5 presents terminal output showing the first five rows of the *incident* table following ETL completion, confirming that incident identifiers, titles, dates, and sponsor country codes have been correctly populated.

```
(base) jingtao@jings-air ~ % psql -h localhost -p 5433 -U jingtao -d coid -c "SELECT incident_id, title, incident_date, sponsor_country FROM incident LIMIT 5;"
incident_id | title | incident_date | sponsor_country
-----
1 | Titan Rain | 2005-08-25 | CHN
2 | Compromise at the State Department | 2006-07-11 | CHN
3 | Compromise of the Pentagon's NIPRNet | 2006-08-17 | CHN
4 | Compromise at U.S. Naval War College | 2006-12-04 | CHN
5 | Compromise of National Defense University | 2007-01-12 | CHN
(5 rows)

(base) jingtao@jings-air ~ % \d+
zsh: command not found: d+
(base) jingtao@jings-air ~ % psql -h localhost -p 5433 -U jingtao -d coid -c "\d+ country" -c "\d+ type" -c "\d+ category" -c "\d+ incident" -c "\d+ incident_target"

Table "public.country"
  Column      | Type          | Collation | Nullable | Default | Storage  | Compression | Stats target | Description
-----
country_alpha3 | character(3)   |           | not null |         | extended |             |             |
country_name   | character varying(100) |           | not null |         | extended |             |             |
Indexes:
    "country_pkey" PRIMARY KEY, btree (country_alpha3)
Referenced by:
    TABLE "incident" CONSTRAINT "incident_sponsor_country_fkey" FOREIGN KEY (sponsor_country) REFERENCES country(country_alpha3) ON DELETE SET NULL
    TABLE "incident_target" CONSTRAINT "incident_target_country_alpha3_fkey" FOREIGN KEY (country_alpha3) REFERENCES country(country_alpha3)
Not-null constraints:
    "country_country_alpha3_not_null" NOT NULL "country_alpha3"
    "country_country_name_not_null" NOT NULL "country_name"
Access method: heap

Table "public.type"
  Column      | Type          | Collation | Nullable | Default | Storage  | Compression | Stats target | Description
-----
type_id       | integer       |           | not null |         | plain    |             |             |
type_name     | character varying(50) |           | not null |         | extended |             |             |
Indexes:
    "type_pkey" PRIMARY KEY, btree (type_id)
    "type_type_name_key" UNIQUE CONSTRAINT, btree (type_name)
Referenced by:
    TABLE "incident_target" CONSTRAINT "incident_target_type_id_fkey" FOREIGN KEY (type_id) REFERENCES type(type_id)
Not-null constraints:
    "type_type_id_not_null" NOT NULL "type_id"
    "type_type_name_not_null" NOT NULL "type_name"
Access method: heap

Table "public.category"
  Column      | Type          | Collation | Nullable | Default | Storage  | Compression | Stats target | Description
-----
category_id   | integer       |           | not null |         | plain    |             |             |
category_name | character varying(50) |           | not null |         | extended |             |             |
Indexes:
    "category_pkey" PRIMARY KEY, btree (category_id)
    "category_category_name_key" UNIQUE CONSTRAINT, btree (category_name)
Referenced by:
    TABLE "incident_target" CONSTRAINT "incident_target_category_id_fkey" FOREIGN KEY (category_id) REFERENCES category(category_id)
Not-null constraints:
    "category_category_id_not_null" NOT NULL "category_id"
    "category_category_name_not_null" NOT NULL "category_name"
Access method: heap

Table "public.incident"
  Column      | Type          | Collation | Nullable | Default | Storage  | Compression | Stats target | Description
-----
incident_id   | integer       |           | not null |         | plain    |             |             |
title         | character varying(300) |           | not null |         | extended |             |             |
incident_date | date          |           | not null |         | plain    |             |             |
description    | text          |           |          |         | extended |             |             |
affiliations  | text          |           |          |         | extended |             |             |
response      | text          |           |          |         | extended |             |             |
sources       | text          |           |          |         | extended |             |             |
sponsor_country | character(3)   |           |          |         | extended |             |             |
Indexes:
    "incident_pkey" PRIMARY KEY, btree (incident_id)
    "idx_incident_date" btree (incident_date)
    "idx_incident_sponsor" btree (sponsor_country)
Foreign-key constraints:
    "incident_sponsor_country_fkey" FOREIGN KEY (sponsor_country) REFERENCES country(country_alpha3) ON DELETE SET NULL
Referenced by:
    TABLE "incident_target" CONSTRAINT "incident_target_incident_id_fkey" FOREIGN KEY (incident_id) REFERENCES incident(incident_id) ON DELETE CASCADE
Not-null constraints:
    "incident_incident_id_not_null" NOT NULL "incident_id"
    "incident_title_not_null" NOT NULL "title"
Access method: heap

Table "public.incident_target"
  Column      | Type          | Collation | Nullable | Default | Storage  | Compression | Stats target | Description
-----
target_id     | integer       |           | not null |         | plain    |             |             |
incident_id   | integer       |           | not null |         | plain    |             |             |
country_alpha3 | character(3)   |           | not null |         | extended |             |             |
type_id       | integer       |           |          |         | plain    |             |             |
category_id   | integer       |           |          |         | plain    |             |             |
victims       | text          |           |          |         | extended |             |             |
Indexes:
    "incident_target_pkey" PRIMARY KEY, btree (target_id)
    "idx_target_country" btree (country_alpha3)
    "idx_target_incident" btree (incident_id)
Foreign-key constraints:
    "incident_target_category_id_fkey" FOREIGN KEY (category_id) REFERENCES category(category_id)
    "incident_target_country_alpha3_fkey" FOREIGN KEY (country_alpha3) REFERENCES country(country_alpha3)
    "incident_target_incident_id_fkey" FOREIGN KEY (incident_id) REFERENCES incident(incident_id) ON DELETE CASCADE
    "incident_target_type_id_fkey" FOREIGN KEY (type_id) REFERENCES type(type_id)
Not-null constraints:
    "incident_target_target_id_not_null" NOT NULL "target_id"
```

Figure 5: Terminal verification of *incident* table contents following ETL completion. The first five records confirm correct population of *incident_id*, *title*, *incident_date*, and *sponsor_country* fields.

4.6 Data Limitations

Two data quality issues in the raw CFR source file are documented here.

First, 28 records carry a multi-state sponsor value in which two or more country names are concatenated in a single cell (e.g., “United Kingdom, United States”). These records cannot be mapped to a single `sponsor_country` foreign key and are assigned a null sponsor value. They represent 1.3 percent of all records.

Second, 24 records carry a null `COUNTRY_ALPHA` value and cannot be assigned to a victim country. These records are excluded from the *incident_target* table, resulting in 2,118 target records rather than 2,142. The excluded records represent 1.1 percent of the raw data.

5 Representative Queries

This section presents ten representative SQL queries executed against the COID database. Each query addresses one of the research questions defined in Section 1 or a directly related analytical sub-question. Together they demonstrate the range of SQL features employed in the project and illustrate the research utility of the normalized schema. All queries are executed in PostgreSQL 18.3.

5.1 Q1: Most Frequently Targeted Countries

The following query returns the ten countries with the highest number of targeting records in *incident_target*, joining to *country* to retrieve full country names. It addresses RQ1 directly.

```
SELECT it.country_alpha3,
       c.country_name,
       COUNT(*) AS incident_count
FROM incident_target it
JOIN country c ON it.country_alpha3 = c.country_alpha3
GROUP BY it.country_alpha3, c.country_name
ORDER BY incident_count DESC
LIMIT 10;
```

The United States is the most frequently targeted country (328 records), followed by China (128), Russia (92), Ukraine (92), and South Korea (85). The dominance of the United States reflects both its global prominence as a target and the likely reporting bias inherent in English-language open-source data collection.

```

psql (18.3 (Postgres.app))
Type "help" for help.

coid=# SELECT it.country_alpha3,
coid=#         c.country_name,
coid=#         COUNT(*) AS incident_count
coid=# FROM incident_target it
coid=# JOIN country c ON it.country_alpha3 = c.country_alpha3
coid=# GROUP BY it.country_alpha3, c.country_name
coid=# ORDER BY incident_count DESC
coid=# LIMIT 10;
 country_alpha3 | country_name | incident_count
-----+-----+-----
USA             | United States |           328
CHN             | China        |           128
RUS             | Russia       |            92
UKR             | Ukraine      |            92
KOR             | South Korea  |            85
IND             | India        |            78
GBR             | United Kingdom |           75
IRN             | Iran         |            74
DEU             | Germany      |            72
ISR             | Israel       |            61
(10 rows)

```

Figure 6: Q1 result: top ten most frequently targeted countries by incident record count.

5.2 Q2: Most Active Sponsor States

This query aggregates incident counts by sponsoring state, joining *incident* to *country* on the *sponsor_country* foreign key.

```

SELECT c.country_name AS sponsor,
       COUNT(*) AS incidents_sponsored
FROM incident i
JOIN country c ON i.sponsor_country = c.country_alpha3
GROUP BY c.country_name
ORDER BY incidents_sponsored DESC
LIMIT 5;

```

China is the most prolific sponsor (270 unique incidents), followed by the Russian Federation (169), Iran (99), North Korea (81), and the United States (13). The counts reflect unique operations rather than raw targeting records, since one operation can produce multiple rows in *incident_target*.

```

coid=# SELECT c.country_name AS sponsor,
coid=#         COUNT(*) AS incidents_sponsored
coid=# FROM incident i
coid=# JOIN country c ON i.sponsor_country = c.country_alpha3
coid=# GROUP BY c.country_name
coid=# ORDER BY incidents_sponsored DESC
coid=# LIMIT 5;

```

sponsor	incidents_sponsored
China	270
Russia	169
Iran	99
North Korea	81
United States	13

(5 rows)

Figure 7: Q2 result: top five sponsor states by unique incident count.

5.3 Q3: Distribution of Operation Types

This query computes the count and percentage share of each operation type using a window function (`SUM() OVER()`) to calculate the denominator without a subquery.

```

SELECT t.type_name,
       COUNT(*) AS count,
       ROUND(COUNT(*) * 100.0 /
             SUM(COUNT(*) OVER (), 1) AS pct
FROM incident_target it
JOIN type t ON it.type_id = t.type_id
GROUP BY t.type_name
ORDER BY count DESC;

```

Espionage is overwhelmingly the dominant operation type (1,775 records, 88.3 percent of typed records), followed by Financial Theft (72 records, 3.6 percent) and Sabotage (61 records, 3.0 percent). This concentration reflects the primary strategic motivation of state-sponsored cyber actors: intelligence collection rather than disruption or destruction.

```

coid=# SELECT t.type_name,
coid=#         COUNT(*) AS count,
coid=#         ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER (), 1) AS pct
coid=# FROM incident_target it
coid=# JOIN type t ON it.type_id = t.type_id
coid=# GROUP BY t.type_name
coid=# ORDER BY count DESC;

```

type_name	count	pct
Espionage	1775	88.3
Financial Theft	72	3.6
Sabotage	61	3.0
Denial of service	40	2.0
Data destruction	39	1.9
Defacement	13	0.6
Doxing	11	0.5

(7 rows)

Figure 8: Q3 result: distribution of operation types by count and percentage share.

5.4 Q4: Distribution of Target Sectors

This query computes the distribution of targeting records across the four victim sector categories.

```

SELECT cat.category_name,
       COUNT(*) AS count,
       ROUND(COUNT(*) * 100.0 /
             SUM(COUNT(*) OVER (), 1) AS pct
FROM incident_target it
JOIN category cat ON it.category_id = cat.category_id
GROUP BY cat.category_name
ORDER BY count DESC;

```

Government (781 records, 37.3 percent) and Private sector (773 records, 37.0 percent) are nearly equally targeted, together accounting for approximately 74 percent of all typed targeting records. Civil society accounts for 361 records (17.3 percent) and Military for 177 records (8.5 percent).

```

coid=# SELECT cat.category_name,
coid=#         COUNT(*) AS count,
coid=#         ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER ()), 1) AS pct
coid=# FROM incident_target it
coid=# JOIN category cat ON it.category_id = cat.category_id
coid=# GROUP BY cat.category_name
coid=# ORDER BY count DESC;

```

category_name	count	pct
Government	781	37.3
Private sector	773	37.0
Civil society	361	17.3
Military	177	8.5

(4 rows)

Figure 9: Q4 result: distribution of targeting records across victim sector categories.

5.5 Q5: Annual Cumulative Incident Trend

This query uses a subquery to first aggregate incident counts by year, then applies a window function (`SUM() OVER (ORDER BY yr)`) to compute a running cumulative total, directly addressing RQ4.

```

SELECT yr,
       yearly_count,
       SUM(yearly_count) OVER (ORDER BY yr) AS cumulative
FROM (
    SELECT EXTRACT(YEAR FROM incident_date)::INT AS yr,
           COUNT(*) AS yearly_count
    FROM incident
    WHERE incident_date IS NOT NULL
    GROUP BY yr
) t
ORDER BY yr;

```

Disclosed incidents grow from 1 unique operation in 2005 to a peak of 147 in 2022, with a notable acceleration after 2019. The cumulative count reaches 776 unique operations by March 2023, with 2020 (119), 2021 (107), and 2022 (147) representing the three highest annual totals in the observation period.

```

coid=# SELECT yr,
coid=#         yearly_count,
coid=#         SUM(yearly_count) OVER (ORDER BY yr) AS cumulative
coid=# FROM (
coid=#     SELECT EXTRACT(YEAR FROM incident_date)::INT AS yr,
coid=#             COUNT(*) AS yearly_count
coid=#     FROM incident
coid=#     WHERE incident_date IS NOT NULL
coid=#     GROUP BY yr
coid=# ) t
coid=# ORDER BY yr;
  yr | yearly_count | cumulative
-----+-----+-----
2005 |           1 |          1
2006 |           3 |          4
2007 |          10 |         14
2008 |           7 |         21
2009 |           4 |         25
2010 |           9 |         34
2011 |          15 |         49
2012 |          12 |         61
2013 |          18 |         79
2014 |          32 |        111
2015 |          34 |        145
2016 |          33 |        178
2017 |          46 |        224
2018 |          80 |        304
2019 |          74 |        378
2020 |         119 |        497
2021 |         107 |        604
2022 |         147 |        751
2023 |          25 |        776
(19 rows)

```

Figure 10: Q5 result: annual and cumulative unique incident counts, 2005–2023.

5.6 Q6: Most Active Attacker–Victim Country Pairs

This query joins three tables (*incident*, *incident_target*, and *country* twice under different aliases) to identify the most frequent bilateral attacker-victim pairings, addressing RQ2.

```

SELECT c_sp.country_name AS sponsor,
       c_vt.country_name AS victim,
       COUNT(*) AS incidents
FROM incident i
JOIN country c_sp
  ON i.sponsor_country = c_sp.country_alpha3
JOIN incident_target it

```

```

ON i.incident_id = it.incident_id
JOIN country c_vt
ON it.country_alpha3 = c_vt.country_alpha3
GROUP BY c_sp.country_name, c_vt.country_name
ORDER BY incidents DESC
LIMIT 10;

```

The China–United States pairing is the most active (133 incidents), followed by China–China (100, reflecting domestic-facing operations or misattribution), Russia–Ukraine (79), Russia–United States (63), and Iran–United States (61). These pairings reflect the dominant geopolitical rivalries in the cyber domain over the 2005–2023 period.

```

coid=# SELECT c_sp.country_name AS sponsor,
coid=#         c_vt.country_name AS victim,
coid=#         COUNT(*) AS incidents
coid=# FROM incident i
coid=# JOIN country c_sp ON i.sponsor_country = c_sp.country_alpha3
coid=# JOIN incident_target it ON i.incident_id = it.incident_id
coid=# JOIN country c_vt ON it.country_alpha3 = c_vt.country_alpha3
coid=# GROUP BY c_sp.country_name, c_vt.country_name
coid=# ORDER BY incidents DESC
coid=# LIMIT 10;

```

sponsor	victim	incidents
China	United States	133
China	China	100
Russia	Ukraine	79
Russia	United States	63
Iran	United States	61
Iran	Iran	46
North Korea	South Korea	43
China	India	39
Russia	Russia	37
Iran	Israel	36

(10 rows)

Figure 11: Q6 result: top ten most active attacker–victim country pairings.

5.7 Q7: China vs. Russia Annual Comparison

This query uses `COUNT(*) FILTER (WHERE ...)` to pivot sponsor-specific incident counts into side-by-side columns by year, enabling direct comparison of the two most active sponsors.

```

SELECT EXTRACT(YEAR FROM i.incident_date)::INT AS yr,
COUNT(*) FILTER (
WHERE i.sponsor_country = 'CHN') AS china,
COUNT(*) FILTER (
WHERE i.sponsor_country = 'RUS') AS russia

```

```

FROM incident i
WHERE i.incident_date IS NOT NULL
      AND i.sponsor_country IN ('CHN', 'RUS')
GROUP BY yr
ORDER BY yr;

```

China maintains consistent activity throughout the observation window, while Russia exhibits sharp increases in 2014 (coinciding with the annexation of Crimea) and again in 2022 (coinciding with the full-scale invasion of Ukraine).

```

coid=# SELECT EXTRACT(YEAR FROM i.incident_date)::INT AS yr,
coid=#         COUNT(*) FILTER (WHERE i.sponsor_country = 'CHN') AS china,
coid=#         COUNT(*) FILTER (WHERE i.sponsor_country = 'RUS') AS russia
coid=# FROM incident i
coid=# WHERE i.incident_date IS NOT NULL
coid=#       AND i.sponsor_country IN ('CHN','RUS')
coid=# GROUP BY yr
coid=# ORDER BY yr;

```

yr	china	russia
2005	1	0
2006	3	0
2007	7	1
2008	5	2
2009	3	0
2010	6	0
2011	9	0
2012	5	0
2013	11	3
2014	17	5
2015	13	9
2016	5	10
2017	6	17
2018	19	25
2019	28	11
2020	37	23
2021	39	15
2022	44	36
2023	8	8

(19 rows)

Figure 12: Q7 result: annual incident counts for China and Russia, 2005–2023.

5.8 Q8: Most Common Operation Type Per Sponsor

This query uses a subquery with `RANK() OVER (PARTITION BY ...)` to identify each major sponsor state's most frequently used operation type, demonstrating the use of window functions for within-group ranking.

```

SELECT sponsor, type_name, cnt
FROM (

```

```

SELECT c.country_name AS sponsor,
       t.type_name,
       COUNT(*) AS cnt,
       RANK() OVER (
         PARTITION BY c.country_name
         ORDER BY COUNT(*) DESC
       ) AS rnk
FROM incident i
JOIN country c
  ON i.sponsor_country = c.country_alpha3
JOIN incident_target it
  ON i.incident_id = it.incident_id
JOIN type t ON it.type_id = t.type_id
GROUP BY c.country_name, t.type_name
) sub
WHERE rnk = 1
ORDER BY cnt DESC
LIMIT 8;

```

Espionage is the dominant operation type for all eight major sponsors returned by the query, including China (712), Russia (325), Iran (299), and North Korea (100). Contrary to what might be expected from open-source reporting on North Korean financial operations, Espionage ranks first for North Korea in this dataset as well, suggesting that the CFR Tracker records more espionage attributions than financial theft attributions for that state.

```

coid=# SELECT sponsor, type_name, cnt
coid=# FROM (
coid(#   SELECT c.country_name AS sponsor,
coid(#       t.type_name,
coid(#       COUNT(*) AS cnt,
coid(#       RANK() OVER (PARTITION BY c.country_name ORDER BY COUNT(*) DESC) AS rnk
coid(#   FROM incident i
coid(#   JOIN country c ON i.sponsor_country = c.country_alpha3
coid(#   JOIN incident_target it ON i.incident_id = it.incident_id
coid(#   JOIN type t ON it.type_id = t.type_id
coid(#   GROUP BY c.country_name, t.type_name
coid(# ) sub
coid=# WHERE rnk = 1
coid=# ORDER BY cnt DESC
coid=# LIMIT 8;

```

sponsor	type_name	cnt
China	Espionage	712
Russia	Espionage	325
Iran	Espionage	299
North Korea	Espionage	100
Vietnam	Espionage	32
Pakistan	Espionage	32
Spain	Espionage	30
United States	Espionage	16

(8 rows)

Figure 13: Q8 result: top-ranked operation type per sponsor state using RANK() OVER (PARTITION BY).

5.9 Q9: Countries Most Targeted in the Government Sector

This query filters *incident_target* records to the Government category and counts victim country occurrences, addressing the sector-specific dimension of RQ3.

```
SELECT c.country_name,
       COUNT(*) AS govt_incidents
FROM incident_target it
JOIN country c
     ON it.country_alpha3 = c.country_alpha3
JOIN category cat
     ON it.category_id = cat.category_id
WHERE cat.category_name = 'Government'
GROUP BY c.country_name
ORDER BY govt_incidents DESC
LIMIT 10;
```

The United States leads in government-sector targeting (109 records), followed by Ukraine (48), China (42), Russia (37), and India (31). Government institutions in major powers and active conflict zones account for a disproportionate share of recorded targeting records.

```
coid=# SELECT c.country_name,
coid=#         COUNT(*) AS govt_incidents
coid=# FROM incident_target it
coid=# JOIN country c ON it.country_alpha3 = c.country_alpha3
coid=# JOIN category cat ON it.category_id = cat.category_id
coid=# WHERE cat.category_name = 'Government'
coid=# GROUP BY c.country_name
coid=# ORDER BY govt_incidents DESC
[coid=# LIMIT 10;
```

country_name	govt_incidents
United States	109
Ukraine	48
China	42
Russia	37
India	31
South Korea	27
United Kingdom	27
Germany	25
Israel	23
Iran	22

(10 rows)

Figure 14: Q9 result: top ten countries by government-sector targeting records.

5.10 Q10: Incident Volume Before and After 2020

This query uses a `CASE WHEN` expression to partition the observation period at 2020 and compare incident volumes across the two sub-periods, providing a simple before-after comparison that contextualizes the acceleration of cyber operations in the COVID-19 era and beyond.

```
SELECT CASE WHEN EXTRACT(YEAR FROM i.incident_date) < 2020
      THEN 'Before 2020'
      ELSE '2020 and after'
END AS period,
COUNT(*) AS incident_count
FROM incident i
JOIN incident_target it
  ON i.incident_id = it.incident_id
WHERE i.incident_date IS NOT NULL
GROUP BY period
ORDER BY period;
```

The pre-2020 period (2005–2019, 15 years) accounts for 1,285 targeting records while the post-2020 period (2020–2023, approximately 3.25 years) accounts for 789 records, implying a substantially higher annualized rate of disclosed incidents in the more recent period (approximately 243 per year versus 86 per year).

```
coid=# SELECT CASE WHEN EXTRACT(YEAR FROM i.incident_date) < 2020
coid=#           THEN 'Before 2020'
coid=#           ELSE '2020 and after'
coid=#           END AS period,
coid=#           COUNT(*) AS incident_count,
coid=#           ROUND(AVG(COUNT(*)) OVER (), 1) AS avg_per_period
coid=# FROM incident i
coid=# JOIN incident_target it ON i.incident_id = it.incident_id
coid=# WHERE i.incident_date IS NOT NULL
coid=# GROUP BY period
coid=# ORDER BY period;
```

period	incident_count	avg_per_period
2020 and after	789	1037.0
Before 2020	1285	1037.0

(2 rows)

Figure 15: Q10 result: incident targeting record counts before and after 2020.

6 Data Distribution and Reproducibility

A normalized relational schema and a query library are necessary but not sufficient conditions for a reproducible research database. Reproducibility requires that the underlying data and infrastructure be

accessible to researchers who were not involved in the original construction of the database. To satisfy this requirement, COID is publicly released as a versioned repository hosted on GitHub at <https://github.com/JingUSA/COID>.

The repository is organized into five subdirectories that correspond to the five components of the project: **data** contains flat-file exports of all five tables; **schema** contains both the DDL script (`create_tables.sql`) and a full PostgreSQL dump (`coid_dump.sql`); **queries** contains the ten representative SQL queries documented in Section 5; **etl** contains a Python utility script used to convert the CSV exports to a portable SQLite database; and **docs** contains a codebook describing every column, controlled vocabulary, and known data quality issue. A `README.md` at the repository root provides installation instructions for three access patterns and a citation template.

The repository is designed to serve three categories of users whose technical environments differ substantially. The first category consists of researchers using statistical software such as R, Python, or Stata, who require tabular data but do not operate a database server. These users can load any of the five CSV files directly from the repository without downloading or installing any additional software. In R, for example, the **incident** table is accessible in a single function call:

```
incident <- read_csv(  
  "https://raw.githubusercontent.com/JingUSA/COID/main/data/incident.csv"  
)
```

The equivalent operation in Python requires only `pandas.read_csv()` with the same URL, and Stata users may use `import delimited` with a direct URL argument. The second category consists of researchers who prefer a self-contained file-based database. For these users, the repository provides `coid.db`, a SQLite database generated from the CSV exports using the `export_sqlite.py` script. SQLite requires no server process and can be queried using DB Browser for SQLite or any standard data science library. The third category consists of researchers operating a PostgreSQL instance who require the full relational environment including foreign key constraints, indexes, and multi-table joins. These users can restore the complete database from `coid_dump.sql` using a single `psql` command, after which all queries in the library execute without modification.

The use of ISO 3166-1 alpha-3 country codes as the primary key for the **country** table ensures that COID joins directly with other cross-national datasets without requiring a crosswalk. V-Dem, the World Values Survey, and most datasets produced under the Correlates of War project either use the alpha-3 standard natively or provide documented mappings to it. This design choice allows COID to serve as a linkable module within larger research pipelines that combine cyber operations data with measures of democracy, institutional trust, or economic development.

All data, schema definitions, and query code are released under the terms documented in the repository. The raw source data originates from the CFR Cyber Operations Tracker, which is publicly available and freely accessible. Researchers using COID in published work are asked to cite both this project and the original CFR source as follows:

Tao, Jing. 2026. "Cyber Operations Incident Database (COID)." GitHub. <https://github.com/JingUSA/COID>

Council on Foreign Relations. 2023. “Cyber Operations Tracker.” <https://www.cfr.org/cyber-operations/>

6.1 Interactive SQL Access

In addition to the downloadable files, COID is accessible as a live, queryable database at the following URL:

```
https://lite.datasette.io/?url=https://raw.githubusercontent.com/JingUSA/COID/main/  
data/coid.db
```

This interface is powered by Datasette Lite, a browser-based SQL environment that loads `coid.db` directly from the GitHub repository. No installation or account is required. Researchers can write arbitrary SQL queries against all five tables and view results immediately in the browser. The ten representative queries documented in Section 5 can be executed directly in this environment, as can any custom query the researcher constructs.

Researchers who prefer a local environment may download `coid.db` from the GitHub repository and query it using Python, R, or any SQLite-compatible tool. Both access paths query the identical underlying data.

7 Conclusion

This project has presented the Cyber Operations Incident Database (COID), a normalized relational database built from the Council on Foreign Relations Cyber Operations Tracker. Starting from a 2,142-row denormalized flat file, the project decomposed the source data into five tables—*country*, *incident*, *type*, *category*, and *incident_target*—enforced referential integrity through primary key, foreign key, and check constraints, and delivered a reproducible ETL pipeline, a ten-query analytical library, and a publicly accessible repository.

The analytical results confirm several patterns consistent with the broader literature on state-sponsored cyber activity. Espionage dominates the operation-type distribution, accounting for 88.3 percent of typed targeting records, which reflects the primary strategic motivation of state actors: intelligence collection rather than sabotage or disruption. China is the most prolific sponsor by unique incident count (270 operations), followed by Russia (169), Iran (99), and North Korea (81). The United States is the most frequently targeted country across all sectors (328 targeting records), while Ukraine’s prominence—particularly in government-sector targeting—mirrors the escalation of Russia-linked operations coinciding with the 2014 Crimea annexation and the 2022 full-scale invasion. The post-2020 period exhibits an annualized incident rate approximately three times higher than the preceding fifteen years, suggesting either a genuine acceleration of disclosed cyber operations or an increase in attribution and reporting capacity, or both.

Two data limitations warrant acknowledgment. First, 28 records carry multi-state sponsor values that cannot be resolved to a single `sponsor_country` foreign key and are assigned null values, representing 1.3 percent of all incidents. Second, 24 records carry null victim country codes and are excluded from *incident_target*, accounting for 1.1 percent of the raw data. These omissions are documented in the repository codebook and do not materially affect the aggregate patterns reported in Section 5.

COID is designed to function as a linkable module within larger cross-national research pipelines. The use of ISO 3166-1 alpha-3 country codes as the primary key for the *country* table allows direct joins with the World Values Survey, V-Dem, and Correlates of War datasets without requiring a crosswalk, making COID particularly well suited to research at the intersection of cyber operations, institutional trust, and democratic accountability—the substantive focus of the author’s broader dissertation project. Future extensions could incorporate additional source trackers to reduce English-language reporting bias, expand the schema to accommodate multi-actor attribution records currently stored as null, and add a temporal granularity column to distinguish operations disclosed in the same calendar year but at different points in the conflict cycle.